

Merge Small to Large

Borworntat Dendumrongkul

IOI Thailand Task Team

Advertisement?

Note: This slide is adapted from my IPST Camp slides. This slide has more topics.

If you are interested, you can watch the video of my past lecture here: YouTube.

Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick
- 5 Merging Data Structure
- 6 Sack Technique
- 7 Jump Land
- 8 Additional Problems

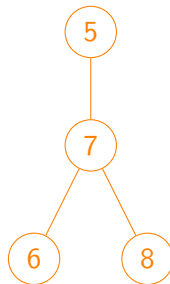
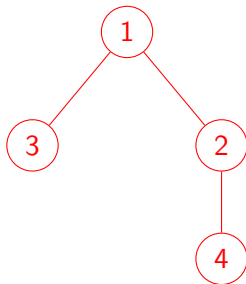
Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick
- 5 Merging Data Structure
- 6 Sack Technique
- 7 Jump Land
- 8 Additional Problems

Disjoint Set Union

Problem: Disjoint Set Union

Are vertices u and v in the same connected component?



Disjoint Set Union

We can just DFS!

```
int a, b;
bool reach_b = false;
void dfs(int u, int p) {
    if(u == b) {
        reach_b = true;
    }
    for(auto v: adj[u]) {
        if(v == p) {
            continue;
        }
        dfs(v, u);
    }
}
// ...
dfs(a, 0);
```

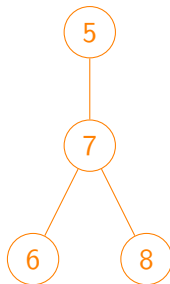
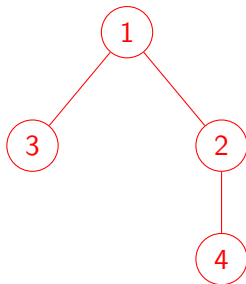
Time Complexity: $\mathcal{O}(V + E)$

Disjoint Set Union

It quite slow... How about changing the adjacency list to an array.

Definition: Parent Array

$\text{parent}[u] :=$ direct parent of vertex u but if u has no parent $\text{parent}[u]$ will store itself.



index	1	2	3	4	5	6	7	8
parent	1	1	1	2	5	7	5	7

Disjoint Set Union

index	1	2	3	4	5	6	7	8
parent	1	1	1	2	5	7	5	7

So, we can easily write `find_root` function to find the root of component.

```
int parent[N];  
int find_root(int u) {  
    if(u == parent[u]) {  
        return u;  
    }  
    return find_root(parent[u]);  
}
```

Time Complexity: $\mathcal{O}(V)$

Disjoint Set Union

index	1	2	3	4	5	6	7	8
parent	1	1	1	2	5	7	5	7

Or we can just use while loop!

```
int parent[N];
int find_root(int u) {
    while(u != parent[u]) {
        u = parent[u];
    }
    return u;
}
```

Time Complexity: $\mathcal{O}(V)$

Disjoint Set Union

It will be very slow if height of the tree is large.

1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9 - 10

By calling `find_root(10)` is **very slow**! So, we have to try to optimize this!

Table of Contents

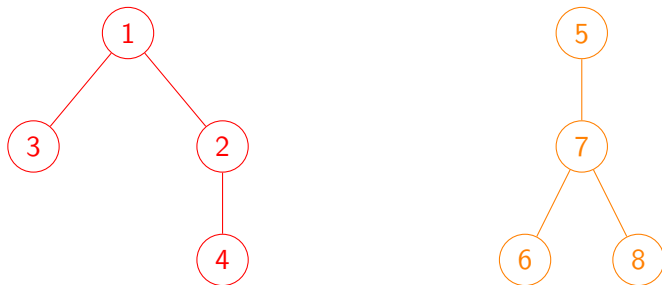
- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression**
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick
- 5 Merging Data Structure
- 6 Sack Technique
- 7 Jump Land
- 8 Additional Problems

Disjoint Set Union: Path Compression

Let's change the definition of the parent array.

Definition: Parent Array

$\text{parent}[u] := \text{root of the component of vertex } u.$



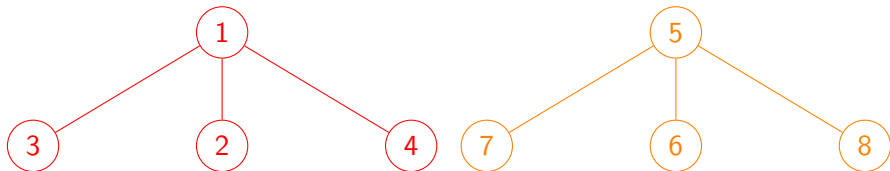
index	1	2	3	4	5	6	7	8
parent	1	1	1	2	5	7	5	7

Disjoint Set Union: Path Compression

Let's change the definition of the parent array.

Definition: Parent Array

$\text{parent}[u] := \text{root of the component of vertex } u.$



index	1	2	3	4	5	6	7	8
parent	1	1	1	1	5	5	5	5

Disjoint Set Union: Path Compression

By adding “memoization” statement.

```
int find_root(int u) {  
    if(u == parent[u]) {  
        return u;  
    }  
    return parent[u] = find_root(parent[u]);  
}
```

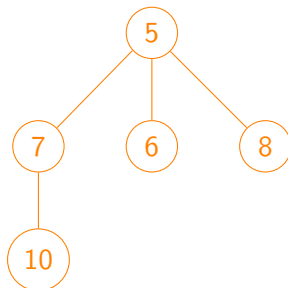
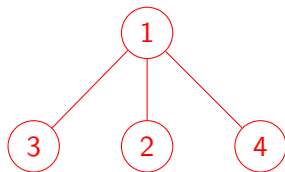
Time Complexity: $\mathcal{O}(\log^*(V))$

Note:

$$\log^*(n) = \begin{cases} 0 & n \leq 1 \\ 1 + \log^*(\log n) & \text{otherwise} \end{cases}$$

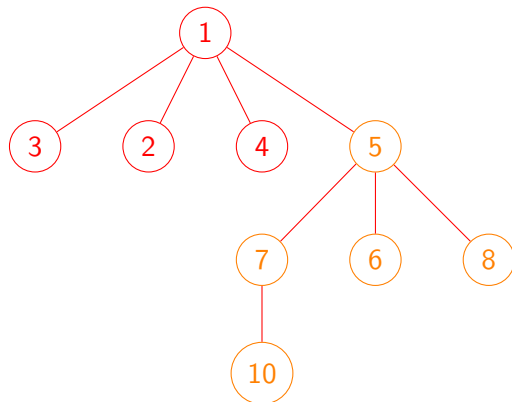
Disjoint Set Union: Path Compression

How about unite?



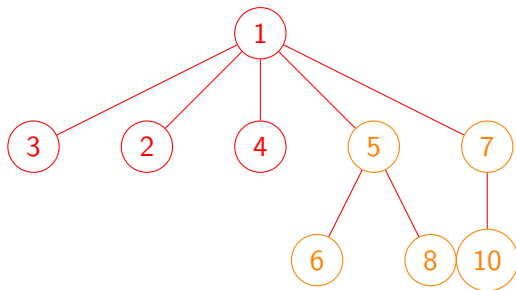
Disjoint Set Union: Path Compression

We can just unite root of the tree with the other one.



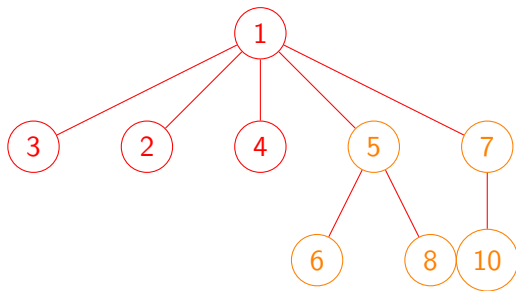
Disjoint Set Union: Path Compression

Call `find_root(7)`



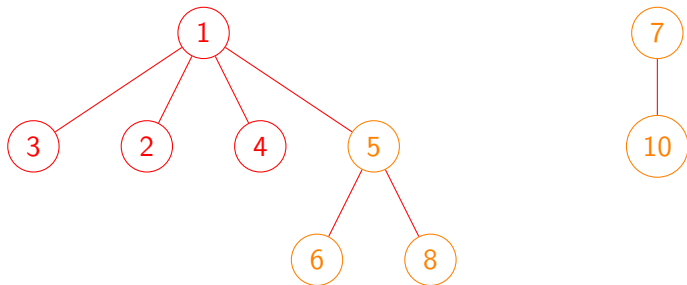
Disjoint Set Union: Path Compression

How about removing the last connection?



Disjoint Set Union: Path Compression

How about removing the last connection?



But the tree is not the same!

Disjoint Set Union: Path Compression

Since path compression does not support removing the rollback, we need to use another approach.

“Disjoint Set Union on Tree”



Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree**
- 4 Amogus Trick
- 5 Merging Data Structure
- 6 Sack Technique
- 7 Jump Land
- 8 Additional Problems

Disjoint Set Union on Tree

We have to maintain the original tree to preserve the ability to rollback the connection.

Let's recall the original parent array definition.

Definition: Parent Array

$\text{parent}[u] :=$ direct parent of vertex u but if u has no parent $\text{parent}[u]$ will store itself.

But... we have to change **how we unite the sets**.

Disjoint Set Union on Tree

There are two main approaches to uniting in the disjoint set union on a tree:

- Union by Rank
- Union by Size

Definition: Rank

In this scope, rank is used to represent the height (or depth) of the tree.

But these two approaches use the same main idea, which is “merge small to large”.

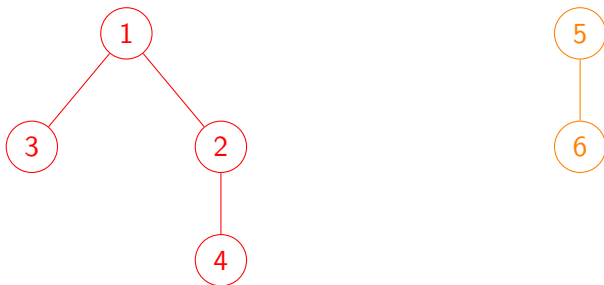
Disjoint Set Union on Tree

And we can recall the old “find_root” function, too.

```
int parent[N];  
int find_root(int u) {  
    while(u != parent[u]) {  
        u = parent[u];  
    }  
    return u;  
}
```

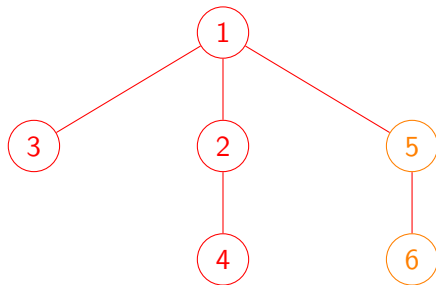

Disjoint Set Union on Tree

First, let's see how “union by rank” works.



Disjoint Set Union on Tree

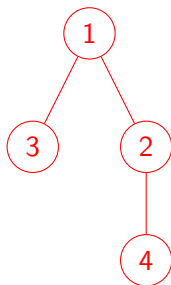
First, let's see how “union by rank” works.



In this case (when the main tree's rank is greater than the merging tree's rank), the rank does not change.

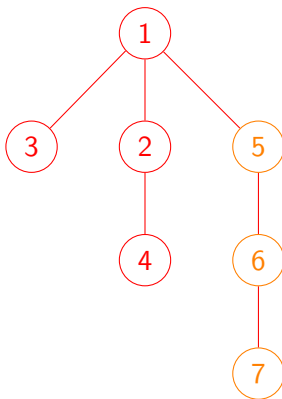
Disjoint Set Union on Tree

When the main tree's rank is equals to the merging tree's rank



Disjoint Set Union on Tree

When the main tree's rank is equals to the merging tree's rank



Rank is increase by one.

Disjoint Set Union on Tree

From observation:

$$\text{rank}_{\text{new}} = \max\{\text{rank}_{\text{main}}, \text{rank}_{\text{merging}} + 1\}$$

```
void unite(int u, int v) {  
    int ru = find_root(u), rv = find_root(rv);  
    if(ru == rv) return ;  
    if(rank[ru] > rank[rv]) swap(ru, rv);  
    rank[rv] = max(rank[rv], rank[ru] + 1);  
    parent[ru] = rv;  
}
```

Disjoint Set Union on Tree

Claim

Union by Rank takes $\mathcal{O}(\log(n))$ time per query.

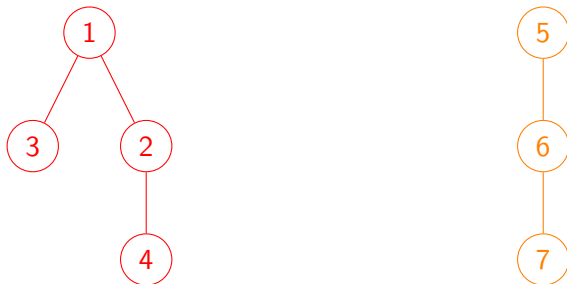
Intuition: By creating tree with

- rank 2 there should be created from tree with 2 rank 1 trees (requires 2 vertices)
- rank 3 there should be created from tree with 2 rank 2 trees (requires 4 vertices)
- ...
- rank n there should be create from tree with 2 rank $n - 1$ trees (requires 2^{n-1} vertices)

We just have n vertices so that the rank of merged tree will not exceed $\log(n)$. So, naive union find will run in $\mathcal{O}(\log(n))$.

Disjoint Set Union on Tree

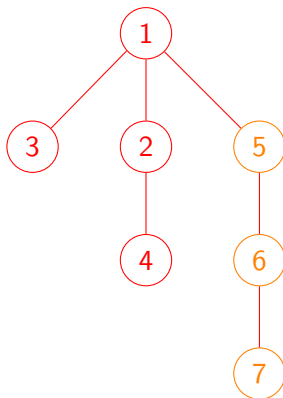
Now, let's see how “union by size” works.



Merge the tree with the smaller size into the larger one.

Disjoint Set Union on Tree

Now, let's see how “union by size” works.



This makes the size of tree bigger.

Disjoint Set Union on Tree

```
void unite(int u, int v) {  
    int ru = find_root(u), rv = find_root(rv);  
    if(ru == rv) return ;  
    if(sz[ru] > sz[rv]) swap(ru, rv);  
    sz[rv] += sz[ru];  
    parent[ru] = rv;  
}
```

Disjoint Set Union on Tree

Claim

Union by Size takes $\mathcal{O}(\log(n))$ time per query.

Intuition: We can prove that each time merging by size

$$\text{rank}_u = \log(n)$$

where n is the size of subtree of u

The rest is union by rank

Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick**
- 5 Merging Data Structure
- 6 Sack Technique
- 7 Jump Land
- 8 Additional Problems

Problem: The Number of Imposters

There are n astronauts in the spacecraft. Each astronaut has to be one of two roles:

- Crewmate
- Imposter

There are m clues from the astronauts (each of which is guaranteed to be true). The clues have only two types:

- u is in the same role as v
- u is not the same role as v

What is the maximum possible number of imposters?

Amogus Trick

We can solve this problem with Disjoint Union on $2n$ vertices.

Definition: Crewmate State

$\text{parent}[1, n] := \text{State of being Crewmate}$

Definition: Imposter State

$\text{parent}[n + 1, 2n] := \text{State of being Imposter}$

index	1	2	3	4	5	6	7	8
parent	1	2	3	4	5	6	7	8

Amogus Trick

Each astronaut i has two vertices:

- $\text{parent}[i]$
- $\text{parent}[i + n]$



Credit: Reddit r/AmongUs

Amogus Trick

There are two cases:

- 1 A and B share the same role
- 2 A and B have different roles

Case 1: A and B share the same role

Unite the two vertices:

- A and B
- $A + n$ and $B + n$

Case 2: A and B have different roles

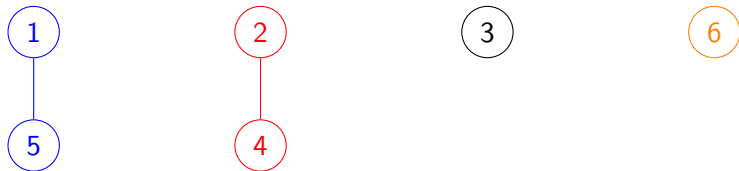
Create special edges by unite the two vertices:

- A and $B + n$
- B and $A + n$

Amogus Trick

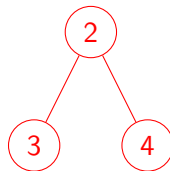
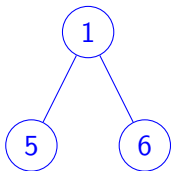
Example:

1. 1 and 2 are different roles



Amogus Trick

- Example:*
1. 1 and 2 are different roles
 2. 2 and 3 are the same role



Definition: Weight

$w[i] :=$ Number of Imposters in the subtree of i

Initially,

- $w[i] = 0$, for $i \leq n$
- $w[i] = 1$, for $i > n$

When we unite two vertices, we need to update the weight:

- $w[A] = w[A] + w[B]$

Such that: The answer is $\max\{w[i], w[i + n]\}$ for some i which is root of the tree.

Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick
- 5 Merging Data Structure**
- 6 Sack Technique
- 7 Jump Land
- 8 Additional Problems

Merging Data Structure

Sometimes, we want to merge 2 data structures together

Problem: Distinct Colors

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \dots, n$ and node 1 is the root. Each node has a color. For each node, determine the number of distinct colors present in its subtree.

Merging Data Structure

Naive Solution: we can DFS and count distinct color of each subtree with sets.

```
void dfs(int u, int p) {  
    for(auto v: adj[u]) {  
        if(v == p) {  
            continue;  
        }  
        dfs(v, u);  
        merge(u, v);  
    }  
}
```

Merging Data Structure

By merging, we can easily do something like:

```
set<int> color[N];  
void merge(int u, int v) {  
    for(auto e: color[v]) {  
        color[u].emplace(e);  
    }  
}
```

The function above has a time complexity $\mathcal{O}(n \log(n))$.

Merging Data Structure

test	verdict	time	
#1	ACCEPTED	0.01 s	»
#2	ACCEPTED	0.01 s	»
#3	ACCEPTED	0.01 s	»
#4	ACCEPTED	0.01 s	»
#5	ACCEPTED	0.01 s	»
#6	TIME LIMIT EXCEEDED	--	»
#7	TIME LIMIT EXCEEDED	--	»
#8	TIME LIMIT EXCEEDED	--	»
#9	TIME LIMIT EXCEEDED	--	»
#10	TIME LIMIT EXCEEDED	--	»
#11	TIME LIMIT EXCEEDED	--	»
#12	ACCEPTED	0.01 s	»
#13	TIME LIMIT EXCEEDED	--	»
#14	TIME LIMIT EXCEEDED	--	»
#15	TIME LIMIT EXCEEDED	--	»

As you can see, it's **TLE** because the running time of the algorithm is $\mathcal{O}(n^2 \log(n))$

Merging Data Structure

We have to use the idea of “merging small to large”. So, we can modify the *merge* function to

```
void merge(set<int> &a, set<int> &b) {  
    if (a.size() < b.size()) {  
        swap(a, b);  
    }  
    for (auto e: b) {  
        a.emplace(e);  
    }  
}
```

By swapping the sets, if we print out the set's size as the answer, the result would be incorrect because the subtrees' sets have been swapped. Therefore, we must maintain the actual size of each set.

Merging Data Structure

By swapping the sets, if we print out the set's size as the answer, the result would be incorrect because the subtrees' sets have been swapped. Therefore, we must maintain the actual size of each set.

```
set<int> color[N];
int sz[N];
void dfs(int u, int p) {
    for(auto v: adj[u]) {
        if(v == p) {
            continue;
        }
        dfs(v, u);
        merge(color[u], color[v]);
    }
    sz[u] = color[u].size();
}
```

Merging Data Structure

test	verdict	time	
#1	ACCEPTED	0.01 s	»
#2	ACCEPTED	0.01 s	»
#3	ACCEPTED	0.01 s	»
#4	ACCEPTED	0.01 s	»
#5	ACCEPTED	0.01 s	»
#6	ACCEPTED	0.38 s	»
#7	ACCEPTED	0.38 s	»
#8	ACCEPTED	0.38 s	»
#9	ACCEPTED	0.41 s	»
#10	ACCEPTED	0.46 s	»
#11	ACCEPTED	0.21 s	»
#12	ACCEPTED	0.01 s	»
#13	ACCEPTED	0.17 s	»
#14	ACCEPTED	0.19 s	»
#15	ACCEPTED	0.38 s	»

YAY, we got **AC**. Why?
(Source Code)

Merging Data Structure

Why “merge small to large” is work in this situation?

Claim

Each element has to move at most $\mathcal{O}(\log(n))$ times.

Proof: When we are trying to merge set A to set B ($|A| \leq |B|$). Then, the new set C 's size is at least $|A| + |A| = 2|A|$. Thus, we move the element e for k times. e will be moved to a set that have size at least 2^k . Since the maximum size of the set is n (at root). So each element will be moved at most $\mathcal{O}(\log(n))$ times.

$$\begin{aligned} n &\geq 2^k \\ \log(n) &\geq k \end{aligned}$$

Running time of the algorithm is $\mathcal{O}(n \log^2(n))$

Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick
- 5 Merging Data Structure
- 6 Sack Technique**
- 7 Jump Land
- 8 Additional Problems

Again...

Problem: Distinct Colors

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \dots, n$ and node 1 is the root. Each node has a color. For each node, determine the number of distinct colors present in its subtree.

Sack Technique

Sack technique can “reuse the data structure” while “merge small into large” technique requires lots of data structure.

Terminology

- “Heavy Child” is a child vertex with the largest subtree size. If there are multiple, pick any.
- “Light Child” is a child vertex which is not a heavy child.
- “Light Edge” is a edge between light child v and its parent u .

Interesting Fact: Each time we move via a light edge, the subtree size decreases by at least half. Since the tree has N nodes, we can take at most $\log(N)$ light edge jumps before reaching the root.

Sack Technique

Heavy child can be easily find by DFS.

```
int sz[N], heavy[N];
void dfs(int u, int p) {
    sz[u] = 1;
    for(auto v: adj[u]) {
        if(v == p) {
            continue;
        }
        dfs(u, v);
        sz[u] += sz[v];
        if(sz[v] > sz[heavy[u]]) {
            heavy[u] = v;
        }
    }
}
```


Sack Technique

After we calculate each subtree's size, we try to solve the problem. By using this algorithm:

```
SACK(u, p, deleting):
    FOR light child v of u:
        SACK(v, u, true)
    IF u is not a leaf:
        SACK(heavy, u, false)
    INSERT(VAL(u))
    FOR light child v of u:
        FOR x in subtree v:
            INSERT(VAL(x))
    // MEMORIZE ANSWER HERE
    IF deleting: // u is light child
        FOR v in subtree u:
            DELETE(VAL(v))
```

Note: To iterate through every vertices in each subtree, there is a famous approach called “Euler Tour Technique”.

Sack Technique

```
SACK(u, p, deleting):  
    FOR light child v of u:  
        SACK(v, u, true)  
    IF u is not a leaf:  
        SACK(heavy, u, false)  
    INSERT(VAL(u))  
    FOR light child v of u:  
        FOR x in subtree v:  
            INSERT(VAL(x))  
    // MEMORIZE ANSWER HERE  
    IF deleting: // u is light child  
        FOR v in subtree u:  
            DELETE(VAL(v))
```

First, we define the function above to solve this problem. The meaning of this function is: “We are considering the subtree rooted at u , which has parent p , and `deleting` is a variable that determines whether the updated values from this subtree need to be deleted or not.”

Sack Technique

```
SACK(u, p, deleting):
    FOR light child v of u:
        SACK(v, u, true)
    IF u is not a leaf:
        SACK(heavy, u, false)
    INSERT(VAL(u))
    FOR light child v of u:
        FOR x in subtree v:
            INSERT(VAL(x))
    // MEMORIZE ANSWER HERE
    IF deleting: // u is light child
        FOR v in subtree u:
            DELETE(VAL(v))
```

We have to delete the updated data of vertex u when u is light child of p . In the other hand, we do not delete the updated data from vertex u when u is the heavy child of p .

Sack Technique

```
SACK(u, p, deleting):
  FOR light child v of u:
    SACK(v, u, true)
  IF u is not a leaf:
    SACK(heavy, u, false)
  INSERT(VAL(u))
  FOR light child v of u:
    FOR x in subtree v:
      INSERT(VAL(x))
  // MEMORIZE ANSWER HERE
  IF deleting: // u is light child
    FOR v in subtree u:
      DELETE(VAL(v))
```

Since we did not delete the heavy child's data, the remaining data to update includes the root of the subtree (u) and the data of all children that are not part of the heavy child's subtree.

Sack Technique

```
SACK(u, p, deleting):
  FOR light child v of u:
    SACK(v, u, true)
  IF u is not a leaf:
    SACK(heavy, u, false)
  INSERT(VAL(u))
  FOR light child v of u:
    FOR x in subtree v:
      INSERT(VAL(x))
  // MEMORIZE ANSWER HERE
  IF deleting: // u is light child
    FOR v in subtree u:
      DELETE(VAL(v))
```

Now that all the data is inserted, we can store the result in an array. Then, if the subtree's data needs to be deleted, we can simply remove it from the data structure.

Sack Technique

```
SACK(u, p, deleting):  
    FOR light child v of u:  
        SACK(v, u, true)  
    IF u is not a leaf:  
        SACK(heavy, u, false)  
    INSERT(VAL(u))  
    FOR light child v of u:  
        FOR x in subtree v:  
            INSERT(VAL(x))  
    // MEMORIZE ANSWER HERE  
    IF deleting: // u is light child  
        FOR v in subtree u:  
            DELETE(VAL(v))
```

In this approach, instead of using set or map as a data structure to count distinct colors, we can just use array to store the data so running time of inserting, erasing, and querying is $\mathcal{O}(1)$.

Sack Technique

```
SACK(u, p, deleting):
    FOR light child v of u:
        SACK(v, u, true)
    IF u is not a leaf:
        SACK(heavy, u, false)
    INSERT(VAL(u))
    FOR light child v of u:
        FOR x in subtree v:
            INSERT(VAL(x))
    // MEMORIZE ANSWER HERE
    IF deleting: // u is light child
        FOR v in subtree u:
            DELETE(VAL(v))
```

(Source Code)

Number of Insertion

- Every vertex u have to be inserted 1 time when we call $SACK(u)$.
- Every time we try to insert x in subtree v which is light child of u . When $SACK(u)$ is called, the subtree size of u have to be at least 2 times of the size of subtree v .

Thus, each vertex will be inserted at most $1 + \lfloor \log(n) \rfloor$

Number of Deletion: Number of deletion of each vertex is always not exceed the number of insertion which is $1 + \lfloor \log(n) \rfloor$

Number of Query: Query function of the data structure is called exactly n times.

Since the query of this data structure is $\mathcal{O}(1)$. So, the running time of the algorithm is $\mathcal{O}(n \log n)$

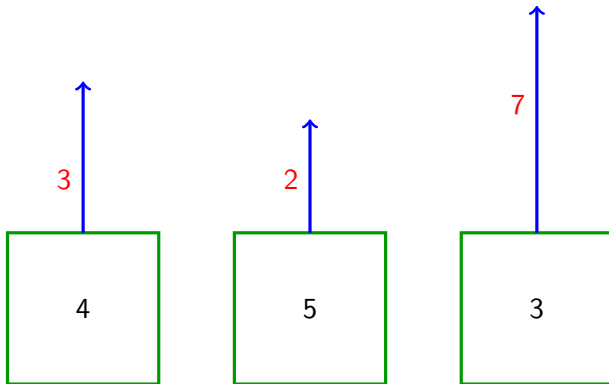
Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick
- 5 Merging Data Structure
- 6 Sack Technique
- 7 Jump Land**
- 8 Additional Problems

Jump Land

Problem: Jump Land

There are $N \times N$ tiles. At the beginning, each tile is at height level h_{ij} and each tile will move up by v_{ij} units. How many tiles are adjacent to each other at the second with the most adjacent tiles?



Jump Land

We can store the time when two tiles are adjacent.

```
struct merge_t {  
    int u, v;  
    double t;  
    merge_t(int _u, int _v, double _t):  
        u(_u), v(_v), t(_t) {}  
};
```

Jump Land

When will tile a and tile b adjacent?

There are 3 cases:

- ① $h_a > h_b$ and $v_a \geq v_b$. In this case, the two tiles will **never** be adjacent.
- ② $h_a > h_b$ and $v_a < v_b$ (wlog). In this case, the two tiles will adjacent at

$$\frac{h_a - h_b}{v_b - v_a}$$

- ③ $h_a = h_b$ and $v_a = v_b$. Two tiles are **always** adjacent.

In the *third* case, we can permanently unite the sets.

Jump Land

Sample Code:

```
vector<merge_t> merge_time;
void add_merge_time(int a, int b) {
    if (v[a] != v[b]) {
        double t = (double) (h[a] - h[b]) / (v[b] - v[a]);
        if (t < 0) {
            return ;
        }
        merge_time.emplace_back(a, b, t);
    }
    else if (h[a] == h[b]) {
        unite(a, b);
    }
}
```

Jump Land

Then we have to sort the `merge_time` by the time when *a* and *b* are adjacent.

```
struct merge_t {  
    int u, v;  
    double t;  
    merge_t(int _u, int _v, double _t):  
        u(_u), v(_v), t(_t) {}  
    bool operator < (const merge_t& o) const {  
        return t < o.t;  
    }  
};  
  
// ...  
  
sort(merge_time.begin(), merge_time.end());
```

Jump Land

At anytime t , we can just **temporary** unite two sets together. After time t , we have to **remove** the connection that was created at time t .

But in **Path Compression** DSU, we cannot remove the connection. So, we have to use **Merge Small to Large** technique to maintain the real forest.

Since we are trying to find size of the component merged. So, we can unite sets by **size**.

```
stack<pair<int , int>> operations;
void rollback() {
    while(!operations.empty()) {
        pair<int , int> op = operations.top();
        operations.pop();
        sz[op.first] -= sz[op.second];
        parent[op.second] = op.second;
    }
}
```

Jump Land

Every time we temporarily unite the sets, we have to store the operations in the stack!

```
void temporary_unite(int a, int b) {  
    int ra = find_root(a);  
    int rb = find_root(b);  
    if(sz[ra] > sz[rb]) {  
        swap(ra, rb);  
    }  
    operations.emplace(rb, ra);  
    sz[rb] += sz[ra];  
    parent[ra] = rb;  
}
```

(Source Code)

Table of Contents

- 1 Disjoint Set Union
- 2 Disjoint Set Union: Path Compression
- 3 Disjoint Set Union on Tree
- 4 Amogus Trick
- 5 Merging Data Structure
- 6 Sack Technique
- 7 Jump Land
- 8 Additional Problems**

Additional Problems

- Stability
- Lion Army
- Split Into Two Sets
- Team-Building

Bibliography I

-  CP-Algorithms (2024). *Disjoint Set Union (Union-Find)*. Accessed: 2025-04-02. URL: https://cp-algorithms.com/data_structures/disjoint_set_union.html.
-  Kleinberg, Jon and Éva Tardos (2006). *Algorithm Design*. Boston: Addison-Wesley. ISBN: 9780321295354.
-  Mahajan, Suneet (2021). *[Tutorial] Amogus Trick*. Accessed: 2025-04-02. URL: <https://codeforces.com/blog/entry/106407>.
-  PourAkhavan, AmirReza (2015). *[Tutorial] Sack (dsu on tree)*. Accessed: 2025-04-02. URL: <https://codeforces.com/blog/entry/44351>.
-  Prasitjutrakul, Somchai (2010). *Algorithm Design and Analysis*. Thailand: Bangkok. ISBN: 978-616-551-194-0.
-  rewhile (2020). *[Explanation] dsu on trees (small to large)*. Accessed: 2025-04-02. URL: <https://codeforces.com/blog/entry/67696>.